

Embedding Neural Sampling and Adversarial Bandit into Gene Expression Programming for Symbolic Regression

Qiang Lu^a, Qiuchen Yuan^a, Dawei Li^b, Congwen Xu^a, Jake Luo^c, Zhiguang Wang^a

^a*Beijing Key Laboratory of Petroleum Data Mining, China University of Petroleum, Beijing, 102249, China*

^b*PetroChina Research Institute of Petroleum Exploration & Development, Beijing, 100083, China*

^c*Department of Health Informatics and Administration, University of Wisconsin Milwaukee, Milwaukee, 53201, United States*

Abstract

Evolutionary computation (EC) methods for symbolic regression (SR) problems use a search method to actively explore the mathematical expression space to find a solution. However, the EC search process does not have a clear direction owing to the randomness of evolutionary operations. In contrast, deep learning methods focus on learning the clear mapping from a given dataset to a mathematical function, but the learning process does not generally involve an active search of the potential solution space. To combine the advantages of the active search of EC and the clear mapping direction of neural networks, this paper proposes a novelty algorithm called GVAE-ABGEP. GVAE-ABGEP incorporates grammar variational autoencoder (GVAE) and adversarial bandit into gene expression programming (GEP) to guide the GEP search process. GVAE-ABGEP partitions the mathematical expression space into many subspaces. It then leverages an adversarial bandit – AvgHExp3 to choose a subspace. In the selected subspace, it then utilizes GVAE autoencoder to sample individuals whose fitnesses around the local optimums, and executes crossover, mutation and selection on EC to explore global optimums. Experiments on 18 SR and 12 physics benchmarks show that GVAE-ABGEP outperforms three baseline GEPs and six baseline machine learning methods.

Keywords: Symbolic Regression, Gene Expression Programming,

Preprint submitted to Applied Soft Computing Journal

August 7, 2024

1. Introduction

Symbolic regression (SR) [?] searches the mathematical expression space Ω to find the function f_θ that best fits a given dataset (X, Y) , i.e., $f_\theta(X) = Y$. SR is a challenging task due to the following two reasons. 1) The search space Ω could be huge, even for rather simple symbolic equations. For example, if a symbolic equation is represented by a binary tree with a maximum depth of 4, 20 variables $(x_1, x_2, \dots, x_{20})$, and 18 basic functions (such as $+$, $-$ and sqrt), the size of the search space is 8.2×10^{162} [?]. 2) The search space Ω could be complex. Ω contains two types of subspaces: the model subspace, and the constant subspace of each model. The model subspace represents the mathematical expressions f without parameters (θ) , such as " $x + \sin(x^2)$ ". For a mathematical expression, e.g., $f_\theta = a_1x + a_2\sin(a_3x^2)$, its constant subspace is the scope of its parameter values $(\theta = [a_1, a_2, a_3])$. In the model subspace, a symbol change in f can lead to a completely different f_θ . While, in f 's constant subspace, if a parameter value changes slightly, f_θ remains almost unchanged, i.e., $\Delta\theta \rightarrow 0, \Delta f_\theta \rightarrow 0$. Moreover, in Ω , there may be many f_θ s to fit a given dataset (X, Y) .

1.1. Related Work

There are two main ways to deal with the SR problem. One is evolutionary computation (**EC**). The other is deep neural networks (**DL**). EC focuses on how to search the space Ω quickly. Evolutionary computation methods, such as genetic programming (GP) [?], gene expression programming (GEP) [?], and Cartesian genetic programming [?], encode an individual as a tree, linear string or graph in Ω .

Then, they utilize the evolutionary operators – crossover, mutation, and selection, to change individuals and generate better offspring to search the Ω space randomly. The main advantage of these EC methods is that they are generalizable and adaptive to various SR problems. However, unlike gradient descent methods used in some machine learning algorithms, these methods search Ω without a clear direction. Moreover, they do not record previous search positions in Ω so that they may visit these positions again. The non-direction and the revisit will degrade the performance of these evolutionary computation methods. The **space partitioning GEP** methods based on

the multi-armed bandit (MAB) are proposed to overcome the two problems [? ?]. They split the model space Ω_m into many subspaces according to the string similarity of mathematical expressions. They then utilize MAB to evaluate these subspaces and search the high-value subspace. However, similar mathematical expressions in each subspace may have very different fitness scores, making it difficult for a MAB method to evaluate subspaces accurately.

On the other hand, neural network-based SR methods focus on training a model to build the mapping between a dataset and a mathematical expression. NN methods contain three mapping types. 1) The first one is " $M : (X, Y) \rightarrow f_\theta$ ". For example, EQL [? ?] tries to build the mapping M through designing a fully-connected neural network where standard activation functions (e.g., $\tanh$, ReLU) are replaced by the basic functions (e.g., "+", " $\times$ ", " $\sin$ "). Once the neural network is trained, it can represent a symbolic equation f_θ fitted to a given dataset. However, EQL relies on a pre-design neural network according to the category of the SR problem (e.g., time series and dynamic system) [?]. 2) The second is " $M_f : (X, Y) \rightarrow f$ ", such as GVAE [? ?], DSR [?], Neural-Guided GP [?], SymbolicGPT [?] and NeSymRes [?]. They utilize the mapping M_f to obtain a mathematical expression f according to the input dataset (X, Y) . They then use the non-linear optimization algorithm – BFGS [?] to get parameters θ in f . 3) The third one is " $M_s : (X, Y) \rightarrow \mathcal{S}$ ", where $\mathcal{S}$ is a group of mathematical symbols ($\{+, -, \times, \ln, \sin, x_0, x_1, \dots\}$) or features. Example algorithms include DBN-SR [?], GNN-SR [?], DeSTrOI [?] and AIFeynman [? ?]. They use the mapping M_s to discover $\mathcal{S}$ from a given dataset (X, Y) . They then employ one of the above EC-based methods to search Ω with regard to the constraint of $\mathcal{S}$. These neural network methods have an excellent ability to discover mathematical equations on specific benchmarks. However, experiments on the large-scale benchmarks [?] indicate that the mathematical equations found by the NN methods are less accurate than those found by the EC-based methods. The reason is that NN methods seem to be more dependent on the training dataset. They could not discover a correct mathematical equation if their NN structures (layer by layer) could not extract valid features from the dataset.

1.2. Motivation

To combine the advantages of the active (generalizable and adaptive) search of EC and the clear mapping direction of neural networks, this paper

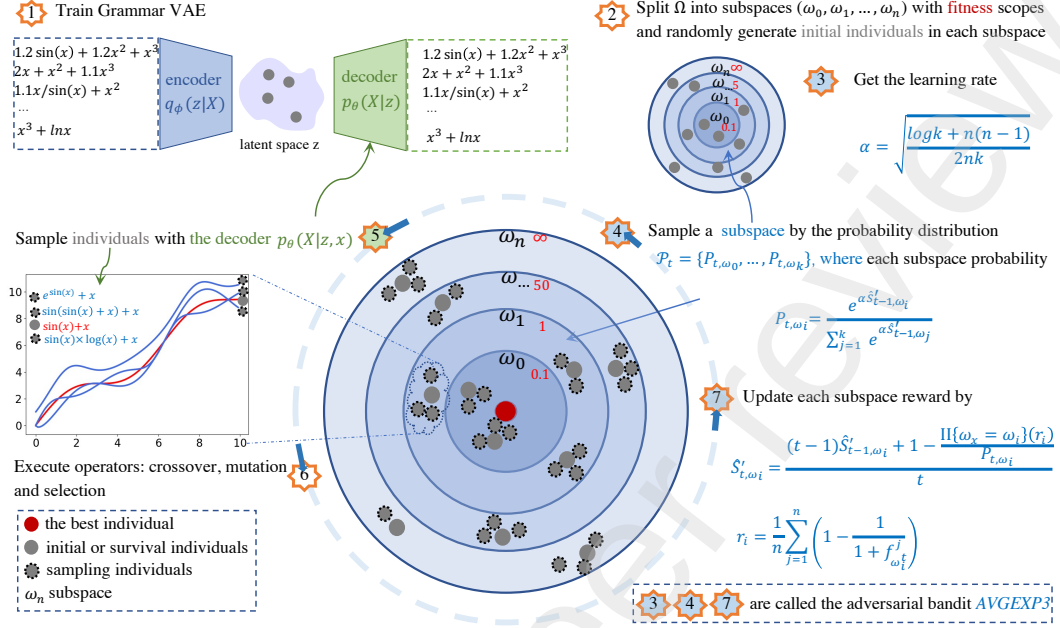


Figure 1: GVAE-ABGEP framework.

proposes a novel GEP method by combining the grammar variational autoencoder (GVAE) and the adversarial bandit (called **GVAE-ABGEP**), as shown in Figure 1. GVAE-ABGEP first trains GVAE to encode individuals into the latent space z where individuals with close fitness are close to each other. It then partitions the space Ω into many subspaces (ω) with fitness scopes. For example, in the subspace ω_0 , the individual fitness distribution samples an individual and its fitness would be in the scope $[0, 0.1)$ with a certain probability. So, each subspace is an individual fitness distribution. As the distribution is unknown, GVAE-ABGEP leverages the adversarial bandit-AvgHExp3 to evaluate these subspaces (distributions) and choose one (ω_i) to explore. It utilizes the trained GVAE to sample individuals with fitness close to a given individual in ω_i . The process of sampling individuals is an exploration on the chosen (local) subspace. After obtaining enough individuals in ω_i , it executes crossover, mutation, and selection to explore the global space Ω .

Unlike the above-mentioned space partitioning methods [? ?] that use string similarities between the mathematical expressions, GVAE-ABGEP partitions the Ω space according to individual fitness. Individuals with sim-

ilar fitness properties can improve the accuracy of AvgHExp3 when it evaluates a subspace. Different from the above-mentioned NN methods, GVAE-ABGEP uses the neural net GVAE to sample individuals, which does not affect the proposed GEP’s generalization and adaption. The reason is that even if GVAE samples randomly, it just degenerates GVAE-ABGEP to ABGEP [?].

The main contributions of this paper are the following:

- We propose a novel method for symbolic regression called GVAE-ABGEP. GVAE-ABGEP is equipped with a grammar variational autoencoder (GVAE), which allows it to explore the local optimal space effectively according to fitness.
- We leverage an adversarial bandit method called AvgHExp3 to evaluate subspaces while maintaining a balance between exploration and exploitation. We prove that AvgHExp3 is an unbiased estimate. Furthermore, we analyze the optimal value of the single parameter (the learning rate α) in AvgHExp3.
- We demonstrate that GVAE-ABGEP significantly outperforms the three baseline GEP methods, **GEP** [?], self-learning GEP (**SL-GEP**) [?] and adversarial bandit GEP (**AB-GEP**) [?] on 18 SR benchmarks. Besides, GVAE-ABGEP surpasses the six baseline machine learning methods, **FFX** [?], linear regression (**LR**), kernel ridge regression (**KR**), random forest regression (**RF**), support vector machine (**SVM**), and **XGBoost** [?] on 12 Feynman benchmarks [?].

The remainder of this paper is organized as follows. In Section 2, we provide details for the background knowledge. Section 3 presents the GVAE-ABGEP algorithm. Sections 4 and 5 report the experimental results and provide discussion, respectively. Finally, we conclude the paper in Section 6.

2. Background

2.1. Grammar Variational Autoencoder

Grammar VAE (GVAE) [?] directly encodes and decodes parse trees to ensure that the generated discrete expression is always syntactically valid, as shown in Figure 2. GVAE learns a coherent latent space $\mathbf{z}$ where nearby points decode similar discrete outputs. To make GVAE learn to map the

discrete expression $\mathbf{X}$ to and from $\mathbf{z}$, GVAE is equipped with an encoder $q_\phi(\mathbf{z} | \mathbf{X})$ and a decoder $p_\theta(\mathbf{X} | \mathbf{z})$.

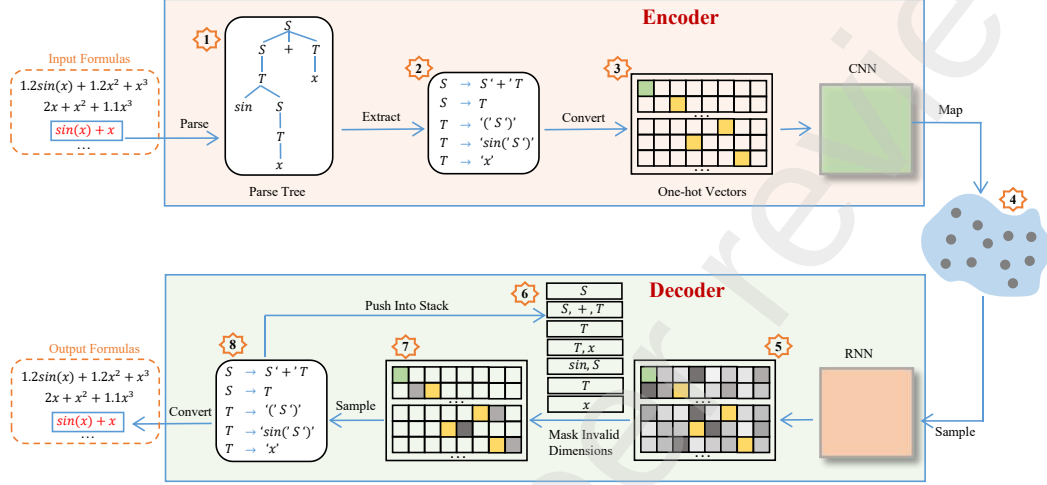


Figure 2: The Encoder and Decoder of GVAE. (1) Parse Tree corresponding to the expression. (2) Grammar rules extracted from the parse tree. (3) One-hot vectors of grammar rules. (4) Latent space $\mathbf{z}$. (5) Unnormalized log probability vectors. (6) Stack of production rule symbols. (7) A set of logarithmic probability vectors. Each dimension corresponds to the grammar rules of the formula. (8) Decoded grammar rules.

The goal of the encoder $q_\phi(\mathbf{z} | \mathbf{X})$ is to map discrete expression $\mathbf{X}$ into a continuous latent vector $\mathbf{z}$. The encoder first converts an expression into a parse tree according to a specific grammar, such as the mathematical expression grammar. It then extracts rules from the parse tree. It generates a one-hot vector for each extracted rule. So, these one-hot vectors form the matrix $\mathbf{M}$. Finally, the encoder utilizes a convolution neural network to map $\mathbf{M}$ into the latent vector $\mathbf{z}$.

The goal of the decoder $p_\theta(\mathbf{X} | \mathbf{z})$ is to map $\mathbf{z}$ back to the discrete expression $\mathbf{X}$. The decoder uses a recurrent neural network to produce a sequence of logit vectors $\mathbf{F}$. To ensure that any sequence of rules is valid, the decoder utilizes a stack to keep track of the state of the parse tree, and uses the symbol popped from the stack to mask out the invalid dimensions of $\mathbf{F}$. Finally, the decoder takes the max value in the dimensions of $\mathbf{F}$ for pointing out rules. These rules could produce the corresponding discrete expression $\mathbf{X}$.

In GVAE, $q_\phi(\mathbf{z} | \mathbf{X})$ is a Gaussian distribution and $p_\theta(\mathbf{z})$ is an isotropic Gaussian prior. To learn the encoder and decoder parameters ϕ and θ , GVAE

maximizes the evidence lower bound (ELBO)

$$\text{ELBO} = \mathbb{E} [\log p_{\theta}(\mathbf{X} | \mathbf{z}) - D_{\text{KL}} [q_{\phi}(\mathbf{z} | \mathbf{X}) \| p_{\theta}(\mathbf{z})]] . \quad (1)$$

The first term ensures that the latent representation $\mathbf{z}$ serves as an accurate encoding of the original data $\mathbf{X}$. The second term encourages the latent distribution $p_{\theta}(\mathbf{z})$ to remain close to the Gaussian distribution $q_{\phi}(\mathbf{z} | \mathbf{X})$. At training time, GVAE samples values from $q_{\phi}(\mathbf{z} | \mathbf{X})$ to compute ELBO and optimizes it via gradient descent.

2.2. Adversarial Bandit

In the basic MAB problem [?], at each round t a player pulls an arm $a_t \in A$ from a finite set of arms and observes the reward r_{t,a_t} corresponding to the selected arm. The goal of MAB is to maximize the cumulative reward after n rounds, $\sum_{t=1}^n r_{t,a_t}$. To maximize the cumulative reward, MAB faces the dilemma of exploration and exploitation, continuously exploiting the current best arm or exploring other arms. Many classical stochastic bandit algorithms are proposed to balance exploration and exploitation, such as ϵ -greedy [?], UCB1 [?], Bayesian bandit [?], and Thompson sampling [?]. These methods have the following two assumptions 1) arms are independent of each other; 2) the rewards associated with each arm are independently and identically distributed (iid). Under these assumptions, stochastic bandit algorithms are shown to be efficient in some static or stable environments. Unlike classical stochastic bandit algorithms, the adversarial bandit abandons these assumptions. It is one of the most generalized bandit problems, as it reacts quickly to the reward change [?].

In a k -armed adversarial bandit environment [?], the environment is called the adversary, and the adversary secretly determines reward $(r_t)_{t=1}^n$ where $r_t \in [0, 1]^k$ is the vector that records k arms' rewards. In each round t , the player generates an action distribution P_t according to the previously observed reward and samples an action a_t from P_t . Then the player receives the reward r_{t,a_t} . To achieve the maximum cumulative reward in an adversarial bandit environment, Exp3 [?] and its variants, such as Exp3-IX [?] and Exp4 [?], are commonly used. Exp3 uses an exponential weight policy to map each arm's cumulative estimated reward into a probability value, denoted as

$$P_{t,i} = \frac{e^{\alpha \hat{S}_{t-1,i}}}{\sum_{j=1}^k e^{\alpha \hat{S}_{t-1,j}}} . \quad (2)$$

Then, Exp3 plays an action randomly according to the probability distribution $\mathcal{P}_t = \{P_{t,1}, \dots, P_{t,k}\}$ in round t . In Equation 2, $\hat{S}_{t-1,i}$ is the cumulative estimated reward of the arm i by the end of round $t - 1$, i.e.,

$$\hat{S}_{t-1,i} = \hat{S}_{t-2,i} + \hat{R}_{t-1,i}, t \geq 2 \quad (3)$$

The estimated reward $\hat{R}_{t,i}$ is computed by the importance weighting [?]]

$$\hat{R}_{t,i} = 1 - \frac{r_{t,a_t}}{P_{t,i}} = 1 - \frac{\mathbb{I}\{a_t = i\} r_{t,a_t}}{P_{t,i}}, \quad (4)$$

where $\mathbb{I}$ is an indicator function that shows which arm is pulled at the round t , denoted as

$$\mathbb{I}\{a_t = i\} = \begin{cases} 1, & \text{if } a_t = i \\ 0, & \text{if } a_t \neq i. \end{cases} \quad (5)$$

3. GVAE-ABGEP Algorithm

GVAE-ABGEP has four main components: GVAE, space partition, subspace selection, and subspace exploration, as shown in Figure 1. GVAE samples individuals with similar fitness and it is used to explore the local optimal subspace. The space partition splits Ω into many subspaces $(\omega_0, \omega_1, \dots, \omega_n)$. The subspace selection leverages the adversarial bandit – AvgHExp3 to choose a subspace by evaluating each subspace. The subspace exploration uses evolutionary operators to search in the chosen subspace.

The proposed GVAE-ABGEP pseudocode is Algorithm 1. The algorithm contains six steps. (1) Train the GVAE model $\mathcal{G}$ (line 1). (2) Split the mathematical expression space Ω into k subspaces $\{\omega_0, \omega_1, \omega_2, \dots, \omega_{k-1}\}$ according to fitness; randomly generate seed individuals in each subspace (lines 2-3). (3) set the learning rate α by Equation 10 based on the number of subspaces k and the number of algorithm iterations n (line 5). (4) In each generation, GVAE-ABGEP leverages a modified adversarial bandit Exp3 – AvgHExp3 to choose a subspace ω_x (lines 7-8). (5) The algorithm samples individuals with $\mathcal{G}$ on seed individuals in ω_x to create the current population (line 9). (6) The algorithm then performs crossover, mutation, and selection to generate the next seed individuals (lines 10-11). (7) The algorithm updates the average estimated reward of each subspace (line 12).

Algorithm 1: GVAE-ABGEP

Input: $k = |\text{subspaces}|, n = |\text{generations}|$

Output: best individual $f(x)$

1 train the GVAE model $\mathcal{G}$

2 partition the mathematical expression space Ω into subspaces $\{\omega_0, \omega_1, \omega_2, \dots, \omega_{k-1}\}$

3 generate seed individuals in each subspace ω_i

4 set $\hat{S}_{0,\omega_i} = 0$ for each ω_i

5 set the learning rate $\alpha = \sqrt{\frac{\log k + n(n-1)}{2nk}}$ (10)

6 **for** $t = 1; t \leq n; t++$ **do**

7 obtain the probability distribution $\mathcal{P}_t = \{P_{t,\omega_1}, \dots, P_{t,\omega_k}\}$ by computing each ω_i probability with the following equation.

$$P_{t,\omega_i} = \frac{e^{\alpha \hat{S}'_{t-1,\omega_i}}}{\sum_{j=1}^k e^{\alpha \hat{S}'_{t-1,\omega_j}}} \quad (9)$$

8 sample a subspace ω_x from $\mathcal{P}_t$ \ \ choose the subspace ω_x

9 sample individuals with $\mathcal{G}$ on seed individuals in ω_x to create the current population

10 perform crossover and mutation on the current population

11 select best individuals as seed individuals

12 update average estimated reward of each ω_i by

$$\hat{S}'_{t,\omega_i} = \frac{\Pi\{\omega_x=\omega_i\} \frac{1}{m} \sum_{j=1}^m \left(1 - \frac{1}{1+f_j^{\omega_x}}\right)}{t} \quad (8)$$

3.1. Training Grammar VAE

To let GVAE $\mathcal{G}$ decode a mathematical expression $\mathbf{s}'$ close to the input mathematical expression $\mathbf{s}$ in terms of fitness, the loss equation of the standard GVAE (Equation 1) is modified to Equation 6.

$$\mathcal{L}(\phi, \theta; \mathbf{s}) = D_{\text{KL}}[q_{\phi}(\mathbf{z} | \mathbf{s}) || p_{\theta}(\mathbf{z})] + \frac{1}{n} \sum_{i=1}^n (\mathbf{s}'(i) - \mathbf{s}(i))^2, \quad (6)$$

where $\mathbf{s}(i)$ ($\mathbf{s}'(i)$) represents the output value of the mathematical expression $\mathbf{s}$ ($\mathbf{s}'$) with input i -th data in the dataset. Since $q_{\phi}(\mathbf{z} | \mathbf{s})$ is a Gaussian distribution, the first term in Equation 6 aims to regularize the latent representation $\mathbf{z}$ so that the latent space is not too complex. The second term ensures that $\mathbf{s}$ and $\mathbf{s}'$ are close in fitness.

After training the GVAE $\mathcal{G}$ with the minimization of the loss equation (Equation 6), the decoder network $q_{\phi}(\mathbf{z} | \mathbf{s})$ is used to sample an individual $\mathbf{s}'$ by the input $\mathbf{s}$.

3.2. Space Partition

GVAE-ABGEP splits the mathematical expression space Ω into k subspaces $\{\omega_0, \omega_1, \dots, \omega_{k-1}\}$ according to the individual (mathematical expression) fitness on the given dataset (X, Y) . Then, GVAE-ABGEP randomly generates some seed individuals in each subspace. For example, shown in Figure 1, given a random individual " $\sin(x) + x$ ", the individual is considered as the seed individual in ω_1 if its fitness in the scope $[0, 1)$.

3.3. Subspace Selection

To maintain the balance between exploration and exploitation on these k subspaces, GVAE-ABGEP leverages a modified adversarial bandit algorithm – AvgHExp3 instead of Exp3 to choose a subspace ω_i because of the two reasons. One is that, according to Equation 3, Exp3 [?] evaluates one subspace ω_i based on the total rewards of all history individuals. Exp3 is sensitive to a large reward, which lets Exp3 choose the subspace where the large reward appears even if the subspace contains many useful small rewards. However, the subspace split by fitness has individuals that share similar fitness. It means the subspace has many similar rewards. So, AvgHExp3 computes the reward r_t by the following equation.

$$r_t = \frac{1}{m} \sum_{j=1}^m \left(1 - \frac{1}{1 + f_{\omega_i^t}^j} \right), \quad (7)$$

where m is the number of individuals in subspace ω_i in the current generation t and $f_{\omega_i^t}^j$ represents the fitness of individual j . The other is that Exp3 using the total estimated reward has a high variance [?]. To avoid the high variance, AvgHExp3 uses the average reward instead of the total reward as the following equation 8.

$$\hat{S}'_{t,\omega_i} = \frac{(t-1)\hat{S}'_{t-1,\omega_i} + 1 - \frac{\Pi\{\omega_t=\omega_i\} \frac{1}{m} \sum_{j=1}^m \left(1 - \frac{1}{1+f_{\omega_i^t}^j}\right)}{P_{t,\omega_i}}}{t} \quad (8)$$

The variance of Equation 8 is less than that of Equation 3 in Exp3. The detailed proof is provided in Appendix A. In the t -th generation, AvgHExp3 first calculates the average estimated reward $\hat{S}'_{t,\omega_i}$ of each subspace according to Equation 8. With Equation 8, the probability of selecting the subspace ω_i is computed by the following importance weight equation.

$$P_{t,\omega_i} = \frac{e^{\alpha \hat{S}_{t-1,\omega_i}}}{\sum_{j=1}^k e^{\alpha \hat{S}_{t-1,\omega_j}}}, \quad (9)$$

where k is the number of subspaces and α is the learning rate parameter.

According to Equation 9, AvgHExp3 can generate the probability distribution $\mathcal{P}_t = \{P_{t,\omega_1}, \dots, P_{t,\omega_k}\}$. It then samples a subspace ω_i as the selected subspace by the probability distribution P_{t,ω_i} .

In Equation 9, α significantly affects the probability distribution P_{t,ω_i} of selecting subspaces. An inappropriate α value will degrade the performance of GVAE-ABGEP in exploring subspaces. The optimal value of α is given according to the upper bound of the expected regret of AvgHExp3, denoted as

$$\alpha = \sqrt{\frac{\log k + n(n-1)}{2nk}}, \quad (10)$$

where n is the number of evolution iterations, and k is the number of subspaces. The detailed analysis of α is provided in Appendix B.

3.4. Subspace Exploration

To search Ω , GVAE-ABGEP utilizes the following two methods, local exploration and global exploration. The local exploration enables GVAE-ABGEP to search in a local optimal space, while global exploration prevents GVAE-ABGEP from getting stuck in a local optimum.

3.4.1. Local Exploration

After GVAE-ABGEP uses AvgHExp3 to choose one subspace ω_i , it first acquires seed individuals with the best solutions recorded in ω_i . GVAE-ABGEP then utilizes GVAE to sample individuals close to these seed individuals. GVAE has learned the coherent latent space z , where two closed points have approximate fitness. GVAE can sample individuals close to a seed individual by decoding the seed individual from the latent space z . For example, as shown in Figure 1 (5), GVAE samples three individuals, " $\sin(\sin(x) + x) + x$ ", " $e^{\sin(x)+x}$ " and " $\sin(x) \times \log(x) + x$ " by decoding the seed individual " $\sin(x) + x$ " in ω_1 . The three sampled individuals are close to the seed individual. Therefore, Sampling individuals by GVAE is to search the local optimal space around each seed individual. This let GVAE-ABGEP converge fast.

3.4.2. Global Exploration

These sampled individuals and seed individuals form a population. GVAE-ABGEP performs crossover and mutation on the population. As crossover and mutation generate offsprings that can appear in the entire space Ω , crossover and mutation are global explorations. Moreover, they prevent GVAE-ABGEP from falling into a local optimum.

3.4.3. Update

After the local and global exploration, the search space consists of the individuals sampled by GVAE and the individuals generated by crossover and mutation. Each individual ind in the search space is compared to each seed individual $sind$ in the subspace that contains the individual ind . If ind is better than $sind$, ind will replace $sind$ and be a seed individual. Besides, these individuals in the search space are used to update the average estimated reward of each subspace according to Equation 8.

4. Experiment

4.1. Benchmarks

We compare GVAE-ABGEP with the three baseline GEP methods using the symbolic regression benchmarks (**SRB**). SRB consists of eighteen SR problems derived from four canonical symbolic regression benchmarks,

Nguyen [?], Korns [?], Keijzer [?], and Vladislavleva [?]. The details of these benchmarks are listed in Table 1.¹ Moreover, we also compare GVAE-ABGEP with the other six machine learning methods using Feynman symbolic regression benchmarks (**FSRB**) [?].

Table 1: Symbolic Regression Benchmarks

problem	name	formula	dataset
F1	Nguyen-4	$x^6 + x^5 + x^4 + x^3 + x^2 + x$	$U[-1, 1, 20]$
F2	Nguyen-3	$x^5 + x^4 + x^3 + x^2 + x$	$U[-1, 1, 20]$
F3	Nguyen-2	$x^4 + x^3 + x^2 + x$	$U[-1, 1, 20]$
F4	Nguyen-1	$x^3 + x^2 + x$	$U[-1, 1, 20]$
F5	Koza-3	$x^6 - 2x^4 + x^2$	$U[-1, 1, 20]$
F6	Koza-2	$x^5 - 2x^3 + x$	$U[-1, 1, 20]$
F7	Nguyen-5	$\sin(x^2) \cos(x) - 1$	$U[-1, 1, 20]$
F8	Nguyen-6	$\sin(x) + \sin(x + x^2)$	$U[-1, 1, 20]$
F9	Nguyen-7	$\ln(x + 1) + \ln(x^2 + 1)$	$U[0, 2, 20]$
F10	Keijzer-1	$0.3x \sin(2\pi x)$	$E[-1, 1, 0.1]$
F11	Vladislavleva-2	$e^{-x} x^3 (\cos x \sin x) (\cos x \sin^2 x - 1)$	$E[0.05, 10, 0.1]$
F12	Keijzer-8	$\sqrt{x}$	$U[0, 4, 20]$
F13	Nguyen-9	$\sin(x) + \sin(y^2)$	$U[0, 1, 20]$
F14	Nguyen-10	$2 \sin(x) \cos(y)$	$U[-1, 1, 100]$
F15	Keijzer-11	$xy + \sin((x - 1)(y - 1))$	$U[-3, 3, 100]$
F16	Korns-2	$\frac{1}{1+x^{-4}} + \frac{1}{1+y^{-4}}$	$E[-5, 5, 0.4]$
F17	Keijzer-12	$x^4 - x^3 + \frac{y^2}{2} - y$	$U[-3, 3, 20]$
F18	Vladislavleva-3	$e^{-x} x^3 (\cos x \sin x) (\cos x \sin^2 x - 1) (y - 5) x$	$E[0.05, 10, 0.1] \ y : E[0.05, 10.05, 2]$

4.2. Performance Metrics on SRB

The three baseline GEP method are **GEP** [?], self-learning GEP (**SL-GEP**) [?] and adversarial bandit GEP (**AB-GEP**) [?], respectively. The detailed parameters of each algorithm are listed in Table 2.

¹ Although the problems (F1-F6) are low-order polynomials in the proposed blacklist of benchmark problems [?], we believe that these problems still play an important role in testing the generality of GP (GEP) searches.

Table 2: Parameters of Four GEP Algorithms

Name	Parameter	Value
GEP	function set	$+, -, \times, \div, \sin, \cos, \ln(n), \exp$
	population size	100
	head length	10
	chromosome length	21
	max generations	1000
	crossover Rate	0.7
	mutation rate	0.1
SL-GEP	function set	$+, -, \times, \div, \sin, \cos, \ln(n), \exp$
	population size	100
	head length	10
	chromosome length	21
	max generations	1000
	number of ADFs	2
	head length of ADFs	3
AB-GEP	function set	$+, -, \times, \div, \sin, \cos, \ln(n), \exp$
	population size	100
	head length	10
	chromosome length	21
	max generations	1000
	crossover Rate	0.7
	mutation rate	0.1
GVAE-ABGEP	number of subspaces	[81,100]
	function set	$+, -, \times, \div, \sin, \cos, \ln(n), \exp$
	population size	100
	head length	10
	chromosome length	21
	max generations	1000
	crossover Rate	0.7
	mutation rate	0.1
	number of subspaces	[81,100]
	learning rate	$\sqrt{\frac{\log k + n(n-1)}{2nk}}$

GVAE-ABGEP and the three baseline GEP algorithms run thirty times on SRB. Table 3 illustrates the root mean square error (RMSE) and the standard deviation of the four algorithms. GVAE-ABGEP obtains the smallest RMSE on half of SRB (9/18), while the number of the smallest RMSE ob-

tained by the three other algorithms is 0, 3 and 7, respectively. Compared with AB-GEP, GVAE-ABGEP adds the local exploration of sampling individuals, and the global exploration of evolutionary operations (crossover and mutation). The local exploration of GVAE-ABGEP improves its performance (RMSE) by sampling individuals around the best result in each subspace. This helps the algorithm to find better solutions in the local area, which can lead to better overall performance.

Table 3: Performance Metrics on SRB

Function	GEP	SL-GEP	AB-GEP	GVAE-ABGEP
F1	0.1615 $\pm$ 0.0192	0.2653 $\pm$ 0.1213	0.1381 $\pm$ 0.0340	0.1243 $\pm$ 0.0522
F2	0.1517 $\pm$ 0.0811	0.0977 $\pm$ 0.0122	0.0705 $\pm$ 0.0000	0.0581 $\pm$ 0.0104
F3	0.0977 $\pm$ 0.0190	1.32E-16 $\pm$ 0.0000	0.0786 $\pm$ 0.0000	0.0188 $\pm$ 0.0192
F4	0.0796 $\pm$ 0.0113	0.0181 $\pm$ 0.0181	0.0682 $\pm$ 0.0000	0.0101 $\pm$ 0.0111
F5	0.0484 $\pm$ 0.0163	0.0655 $\pm$ 0.0282	0.0321 $\pm$ 0.0000	0.0797 $\pm$ 0.0140
F6	0.0737 $\pm$ 0.0172	0.0694 $\pm$ 0.0101	0.0565 $\pm$ 0.0000	0.0324 $\pm$ 0.0220
F7	0.0330 $\pm$ 0.0072	0.0810 $\pm$ 0.0003	0.0258 $\pm$ 0.0000	0.0561 $\pm$ 0.0251
F8	0.0854 $\pm$ 0.0316	0.2150 $\pm$ 0.0236	0.0538 $\pm$ 0.0000	0.1430 $\pm$ 0.0833
F9	0.1069 $\pm$ 0.0529	0.1226 $\pm$ 0.0372	0.0537 $\pm$ 0.0000	0.1006 $\pm$ 0.0592
F10	0.0897 $\pm$ 0.0001	0.0913 $\pm$ 0.0043	0.1115 $\pm$ 0.0056	0.0894 $\pm$ 0.0002
F11	0.3179 $\pm$ 0.0003	0.3164 $\pm$ 0.0017	0.3169 $\pm$ 0.0001	0.3164 $\pm$ 0.0004
F12	0.2093 $\pm$ 0.0624	0.2299 $\pm$ 0.0883	0.1420 $\pm$ 0.0269	0.0799 $\pm$ 0.0344
F13	0.0562 $\pm$ 0.0563	0.0702 $\pm$ 0.0275	0.0012 $\pm$ 0.0012	0.0815 $\pm$ 0.0815
F14	0.0689 $\pm$ 0.0689	0.0767 $\pm$ 0.0706	0.0145 $\pm$ 0.0145	0.0659 $\pm$ 0.0659
F15	0.6328 $\pm$ 0.0197	2.4499 $\pm$ 0.1982	0.6267 $\pm$ 0.0016	0.6153 $\pm$ 0.0232
F16	0.2856 $\pm$ 0.0293	0.3684 $\pm$ 0.0541	0.2064 $\pm$ 0.0771	0.1322 $\pm$ 0.0573
F17	5.0377 $\pm$ 1.4872	3.2317 $\pm$ 1.3783	4.3846 $\pm$ 1.1260	4.0022 $\pm$ 1.7931
F18	0.7186 $\pm$ 0.0893	1.0190 $\pm$ 0.0137	0.6512 $\pm$ 0.0109	0.9754 $\pm$ 0.0604
RMSE(Std)	0(3)	3(1)	7(15)	9(0)

The global exploration executes crossover, mutation, and selection in GP (GEP), making GVAE-ABGEP uncertain. So, GVAE-ABGEP has a higher stand derivation (Std) than AB-GEP, as shown in Table 3 and Figure 3. The higher stand derivation lets GVAE-ABGEP could find better results than AB-GEP. Moreover, compared with the other two algorithms – SLGEP and AB-GEP, GVAE-ABGEP has a closed Std to GEP on most of the problems (F3-F11, F13-F15 and F17-F18). The reason is that they have the same evolutionary operations (crossover, mutation, and selection). Even if GVAE is invalid, GVAE-ABGEP will degenerate into GEP. Therefore, the global exploration keeps GVAE-ABGEP generalizable and adaptive searches.

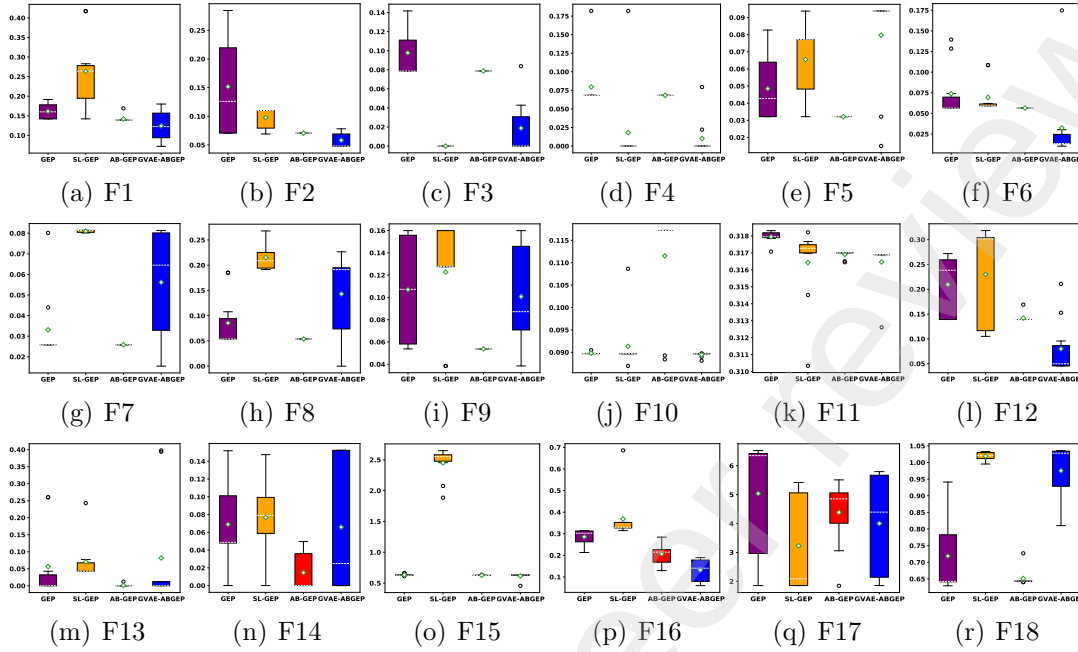


Figure 3: Fitness comparison on all SRB benchmarks.

Table 4 and Figure 4 show the results of the Wilcoxon signed-rank test [?] on comparing GVAE-ABGEP with the three baseline algorithms. The results show that GVAE-ABGEP outperforms them on most of SRB. According to results in Table 4, GVAE-ABGEP is better or equal to the three baseline algorithms on eleven benchmarks (F1, F2, F4, F6, F10-F12 and F14-F17). On the most of benchmarks of low-order polynomials (F1-F6 and F17), GVAE-ABGEP can beat the three baseline algorithms (F1-F2, F4, F6 and F17). While, on half of the benchmarks of trigonometric polynomials (F7-F8, F10-F11, F13-F15 and F18), it obtains better results than the three baseline algorithms (F10-F11 and F14-F15). On two of the other three benchmarks (F9, F12 and F16), it is better than them (F12 and F16).

However, GVAE-ABGEP does not surpass the three baseline GEP algorithms in a few benchmarks. For example, AB-GEP obtains better and more stable results than GVAE-ABGEP on the two benchmarks, F8 and F9, as shown in Figure 3. The reason is that GVAE affects the performance of GVAE-ABGEP. If GVAE samples individuals which are not around a seed individual but in the other subspaces, it means that the subspace selection strategy – AvgHExp3 is invalid. The invalid AvgHExp3 could degrade the

Table 4: Wilcoxon Sign Rank Test

Function	GEP/GVAE-ABGEP		SL-GEP/GVAE-ABGEP		AB-GEP/GVAE-ABGEP	
	P Value	RESULT	P Value	RESULT	P Value	RESULT
F1	0.01367	+	0.00097	+	0.13769	≈
F2	0.00488	+	0.00424	+	0.01855	+
F3	0.00195	+	0.01367	−	0.00195	+
F4	0.00195	+	0.20243	≈	0.00195	+
F5	0.01855	−	0.07665	≈	0.00488	−
F6	0.03222	+	0.03222	+	0.04199	+
F7	0.01640	−	0.01124	+	0.00683	−
F8	0.24609	≈	0.00976	+	0.01367	−
F9	0.45284	≈	0.21582	≈	0.00488	−
F10	0.01367	+	0.18750	≈	0.00488	+
F11	0.00097	+	0.18750	≈	0.16113	≈
F12	0.00195	+	0.00097	+	0.00683	+
F13	0.07568	≈	0.88330	≈	0.00463	−
F14	0.36669	≈	0.28466	≈	0.05493	≈
F15	0.21191	≈	0.00024	+	0.63330	≈
F16	0.00024	+	0.00024	+	0.00122	+
F17	0.03327	+	0.15327	≈	0.23486	≈
F18	0.00073	−	0.06469	≈	0.00024	−
+		10		8		7
−		3		1		6
≈		5		9		5

performance of GVAE-ABGEP. While GVAE-ABGEP could degenerate, it is not the worst algorithm in each benchmark. Figure 4 illustrates that GVAE-ABGEP is better than at least one of the three baseline algorithms on every benchmark. For example, on the benchmark $F7$, although GVAE-ABGEP does not surpass GEP and AB-GEP, it outperforms SL-GEP. So, although the invalid AvgExp3 could degrade the performance of GVAE-ABGEP, the degradation is limited.

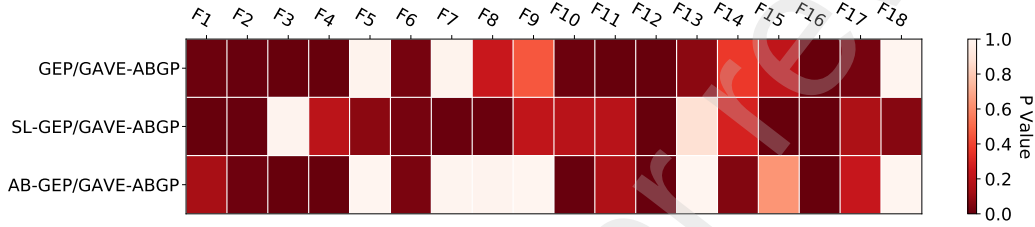


Figure 4: Wilcoxon Signed-Rank Test between GVAE-ABGEP and the three baseline GEP algorithms. The significance level α is 0.05. The darker color indicates a smaller P value, which means that GVAE-ABGEP is significantly better than the three baseline GEP algorithms.

4.3. Performance Metrics on FSRB

The six machine learning methods are **FFX** [?], linear regression (**LR**), kernel ridge regression (**KR**), random forest regression (**RF**), support vector machine (**SVM**), and **XGBoost** [?]. The detailed parameters of each algorithm are tuned according to Table 5.

Table 6 illustrates RMSE results of GVAE-ABGEP and the other six machine learning (**ML**) methods on FSRB. GVAE-ABGEP outperforms these ML methods and obtains the smallest RMSE results on half of the benchmarks (6/12). Moreover, the four RMSE results are zero on benchmarks, $F21(\mu \times N_n)$, $F22(q_2 \times E_f)$, $F26(\omega/c)$ and $F29(\epsilon \times E_f^2)$, which means that GVAE-ABGEP can find the best result (zero) at every run.

Figure 5 illustrates that GVAE-ABGEP obtains more accurate and stable results than the other six ML methods. However, GVAE-ABGEP gets RMSE results close to XGBoost. Especially, GVAE-ABGEP does not surpass XGBoost on 5 benchmarks ($F19$, $F20$, $F25$, $F27$ and $F30$), as shown in Table 6. These benchmarks, $F19(\frac{e^{-\theta^2/2}}{\sqrt{2 \times \pi}})$, $F20(\frac{e^{-(\theta/\sigma)^2/2}}{(\sqrt{2 \times \pi} \times \sigma)})$, $F25(\arcsin(n \times \sin(\theta_2)))$, $F27((h/(2 \times \pi)) \times \omega)$ and $F30(n \times (h/(2 \times \pi)))$ are more complexity and has more operators (+, -, $\times$, $\sin$, $\exp$, ...) than the above benchmarks ($F21$, $F22$,

Table 5: Parameters of Six Machine Learning Algorithms

Name	Parameter	Value
FFX	None	
LR	Normalize	FALSE
KR	Kernal	'linear', 'poly', 'rbf', 'sigmoid'
	Gamma	0.01,0.1,1,10
	Regularization	0.001,0.1,1
RF	Number of Estimators	10, 100, 1000
	Max Features	'sqrt','log2',None
SVM	Kernal	'linear', 'poly', 'rbf', 'sigmoid', 'precomputed'
XGBoost	Learning Rate	0.0001,0.01, 0.05, 0.1, 0.2
	Gamma	0,0.1,0.2,0.3,0.4

Table 6: performance metrics on FSRB

Function	FFX	LR	KR	RF	SVM	XGBoost	GVAE-ABGEP
	RMSE	RMSE	RMSE	RMSE	RMSE	RMSE	RMSE
F19 I.6.2a	0.3044	0.0212	0.0283	0.0113	0.0771	0.0011	0.0041
F20 I.6.2	0.1937	0.0167	0.0038	0.0026	0.0343	0.0010	0.0127
F21 I.12.1	0.2363	1.4143	0.0086	0.3157	0.3371	0.0157	0.0000
F22 I.12.5	0.2265	1.4301	0.0090	0.3248	0.3430	0.0158	0.0000
F23 I.14.4	0.2257	4.8804	0.0025	0.8526	2.1924	0.0399	0.1140
F24 I.25.13	0.1817	0.3584	0.0906	0.0690	0.0799	0.0052	0.0039
F25 I.26.2	0.1709	0.2095	0.0374	0.0218	0.0660	0.0036	0.0151
F26 I.29.4	0.1810	0.3685	0.0858	0.0685	0.0856	0.0053	0.0000
F27 I.34.27	0.2238	0.2293	0.0014	0.0514	0.0660	0.0047	0.0080
F28 I.39.1	0.2307	2.1527	0.0136	0.4842	0.6682	0.0202	0.0017
F29 II.27.18	0.2238	9.5658	0.0051	1.7310	8.4967	0.0491	0.0000
F30 III.12.43	0.2307	0.2295	0.0014	0.0519	0.0662	0.0049	0.0081
	0	0	3	0	0	3	6

F26 and F29). This shows that GVAE-ABGEP is slightly weaker than XGBoost in the face of benchmarks generated by complex functions. In general, the more complex the function, the more complex the data features that it generates. Moreover, GVAE may not capture these features accurately, generating some error samples. These error samples affect the GVAE-ABGEP performance, but the affection is limited because GVAE-ABGEP can still obtain better results than one or several ML methods.

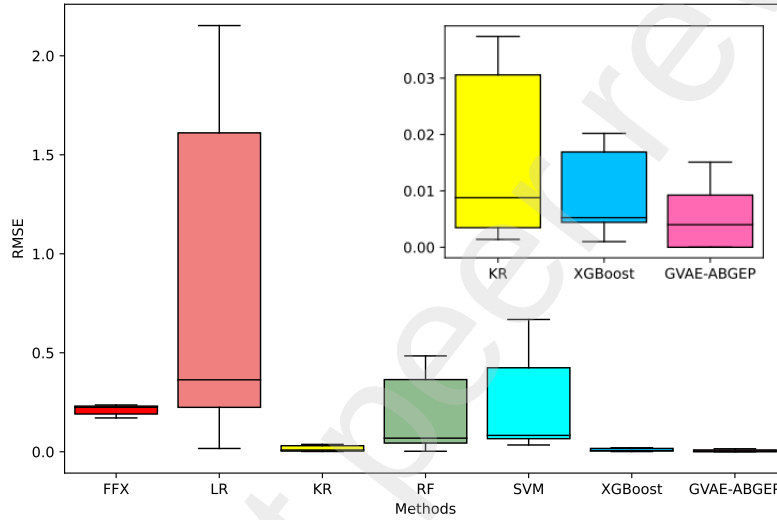


Figure 5: Fitness comparison on FSRB

Figure 6 (the first column) shows that GVAE-ABGEP outperforms the six ML methods on the Wilcoxon signed-rank test. This is because GVAE-ABGEP focuses on searching for fitted formulas in the mathematical expression space, while ML methods focus on finding fitted parameters in a predefined model. The predefined model can impact the RMSE results found by ML methods. Although GVAE is also a predefined model in GVAE-ABGEP, it is used as a local heuristic method to speed up the search process. It does not influence the global search, which is determined by evolutionary operators and the adversarial bandit, AvgHExp3. Therefore, GVAE-ABGEP has a performance advantage in overall benchmarks.

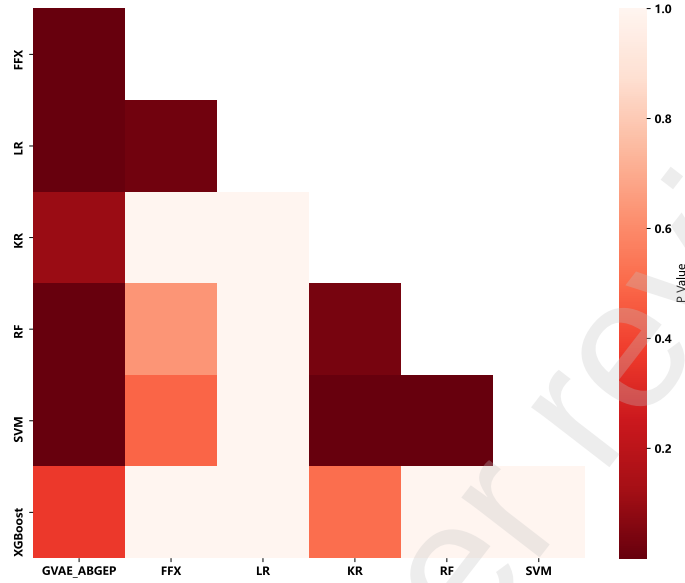


Figure 6: Wilcoxon Signed-Rank Test on FSRB. The significance level α is 0.05. The darker color indicates a smaller P value.

5. Discussion

5.1. Individual Fitness Distribution

The local exploration of GVAE-ABGEP improves its overall performance by enabling it to search in the local optimal space based on individual fitness. This means that GVAE-ABGEP is more likely to find individuals with good fitness in the local area. Compared to the three baseline GEPs, such as ABGEP, GVAE-ABGEP generates more individuals in the local optimal space, as shown in Figure 7. These additional individuals increase the probability of GVAE-ABGEP finding the best solution (the individual with the best fitness).

The global exploration and the subspace selection strategy (AvgHExp3) can help GVAE-ABGEP avoid falling into a local optimum by generating some individuals with poor fitness, as shown by the blue points far from the circle's center in Figure 7. The global exploration uses GEP's operations, crossover and mutation, to change the search space. However, as these operations run, individuals tend to become similar (premature convergence) [? ?] [43, 44], and may still fall into a local optimal space. Therefore, the subspace selection strategy, armed with the adversarial bandit AvgHExp3,

allows GVAE-ABGEP to select different subspaces to explore [?], preventing premature convergence.

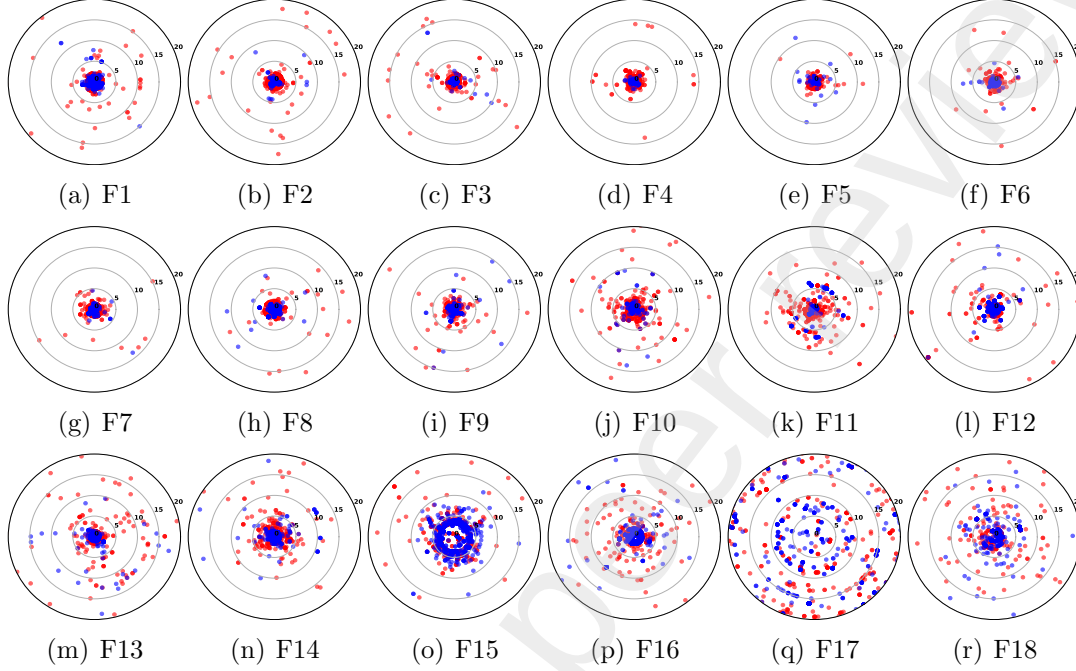


Figure 7: Individual Fitness distribution on all SRB benchmarks. A blue (or red) point represents the fitness of a GVAE-ABGEP (or an AB-GEP) individual recorded every 100 generations. The closer the fitness is to the circle’s centre, the better the fitness.

5.2. Convergence Comparison

Compared with the three baseline GEPs, GVAE-ABGEP converges faster at its initial running stage, as shown in Figure 8(a-d and h). The reason is that GVAE samples individuals around the seed individuals (individuals with the best fitness). These sampled individuals are the local exploration in the local optimal subspace. Moreover, they make GVAE-ABGEP converge faster.

If GVAE cannot sample individuals around seed individuals, the local exploration and the subspace strategy (AvgHExp3) could be invalid. Moreover, GVAE-ABGEP would be degraded into the GEP with more random individuals at every generation. For example, as shown in Figure 8(e)(f), GVAE-ABGEP converges slower than AB-GEP and GEP. Although GVAE-ABGEP converges slowly, it is not easily falling into a local optimum because

it has more random individuals than GEP. So, it still keeps running toward convergence, as shown in Figure 8(f).

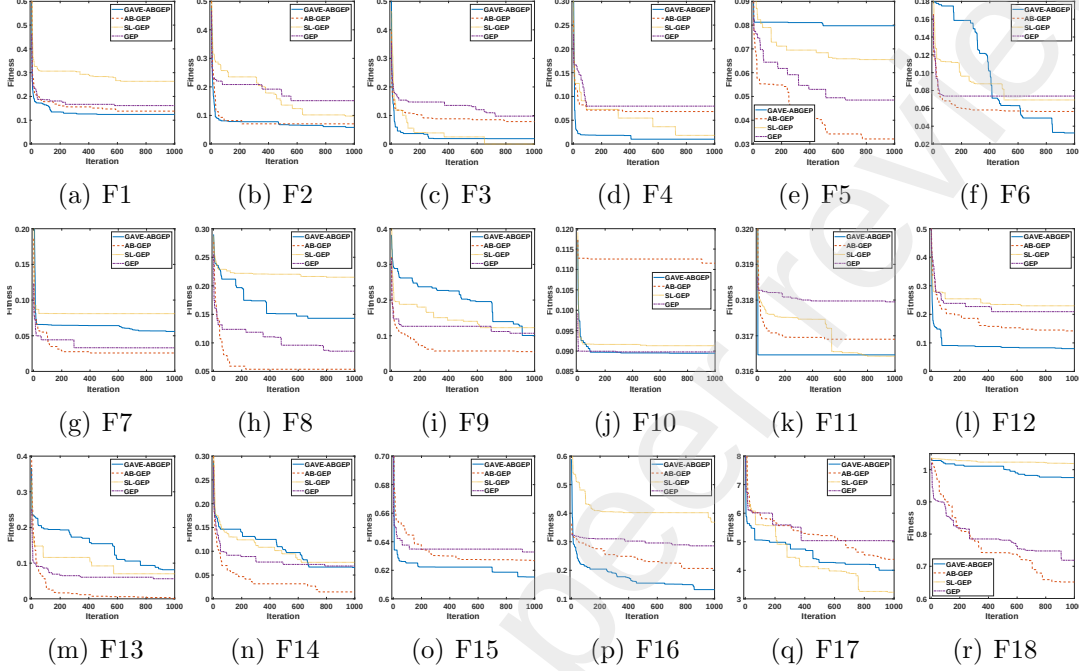


Figure 8: Convergence comparison on all SRB benchmarks.

6. Conclusion

This paper proposes a novel GEP algorithm called GVAE-ABGEP. GVAE-ABGEP combines two kinds of search mechanisms to enhance performance in discovering the best solution in a mathematical expression space. The local exploration search utilizes GVAE to sample individuals in the local optimal space, which leads to faster convergence of GVAE-ABGEP. While the global exploration search leverages AvgHExp3 to balance between the exploration and the exploitation on subspaces. The global exploration also uses the two evolutionary operations, crossover and mutation, to explore the entire mathematical expression space, which prevents GVAE-ABGEP from falling into a local optimum.

The proposed GVAE-ABGEP algorithm combines elements of evolutionary computation and deep learning. GVAE-ABGEP is a novel and innovative approach to symbolic regression problems. The combination of these

two techniques has the potential to make a significant contribution to the symbolic regression field.

The algorithm design and evaluation results indicate that GVAE is a critical component of GVAE-ABGEP. If GVAE cannot sample individuals in a local optimal space, GVAE-ABGEP would degrade into a classic GEP with many random individuals at every iteration. Therefore, our future work will involve finding a better sampling component to replace GVAE.

7. Acknowledgments

This work is supported by China National Key Research Project (No.2019YFC0312003)

Appendix A. The AvgHExp3 Mean and Variance

According to [?], Exp3 uses the total estimated reward $\hat{S}_{t-1,i}$ to evaluate the reward of arms. Although $\hat{S}_{t-1,i}$ is an unbiased estimator, its variance is very high. To decrease variance, AvgHExp3 uses the average estimated reward $\hat{S}'_{t-1,\omega_i}$ to replace $\hat{S}_{t-1,i}$ according to Theorem 1.

Theorem 1. *The average estimated reward $\hat{S}'_{t-1,\omega_i}$ used by AvgHExp3 is an unbiased estimator, and its variance is $1/(t-1)^2$ of Exp3's, i.e., $D[\hat{S}'_{t-1,\omega_i}] = \frac{1}{(t-1)^2} D[\hat{S}_{t-1,i}]$.*

Proof. According to the definition of $\hat{S}'_{t-1,\omega_i}$, i.e., $\hat{S}'_{t-1,\omega_i} = \frac{1}{t-1} \sum_{s=1}^{t-1} \hat{R}_{s,\omega_i}$, where $\hat{R}_{s,\omega_i}$ is the estimated reward of ω_i in iteration t . The expectation of $\hat{S}'_{t-1,\omega_i}$ is

$$E[\hat{S}'_{t-1,\omega_i}] = E\left[\frac{1}{t-1} \sum_{s=1}^{t-1} \hat{R}_{s,\omega_i}\right] = \frac{1}{t-1} \sum_{s=1}^{t-1} E[\hat{R}_{s,\omega_i}]. \quad (\text{A.1})$$

Since $\hat{R}_{s,\omega_i}$ is an unbiased estimator [?], $E[\hat{R}_{s,\omega_i}] = R_{s,\omega_i}$. So $E[\hat{S}'_{t-1,\omega_i}] = \frac{1}{t-1} \sum_{s=1}^{t-1} R_{s,\omega_i}$, which means $\hat{S}'_{t-1,\omega_i}$ is an unbiased estimator of the average actual reward in the subspace ω_i .

The variance of $\hat{S}'_{t-1,\omega_i}$ is

$$\begin{aligned}
D[\hat{S}'_{t-1,\omega_i}] &= E\left[\left(\hat{S}'_{t-1,\omega_i}\right)^2\right] - E^2[\hat{S}'_{t-1,\omega_i}] \\
&= E\left[\left(\frac{1}{t-1} \sum_{s=1}^{t-1} \hat{R}_{s,\omega_i}\right)^2\right] - \left(\frac{1}{t-1} \sum_{s=1}^{t-1} R_{s,\omega_i}\right)^2 \\
&= \frac{1}{(t-1)^2} \left(E\left[\left(\sum_{s=1}^{t-1} \hat{R}_{s,\omega_i}\right)^2\right] - \left(\sum_{s=1}^{t-1} R_{s,\omega_i}\right)^2 \right) \\
&= \frac{1}{(t-1)^2} D[\hat{S}_{t-1,i}]
\end{aligned} \tag{A.2}$$

□

The theorem shows that, compared with Exp3, AvgHExp3 that uses the average estimated reward can decrease the Exp3's variance as the number (t) of GVAE-ABGEP running iterations increases.

Appendix B. Optimal Value of Parameter α

The parameter $\alpha \in [0, +\infty]$ in AvgHExp3 significantly affects the probability distribution $\mathcal{P}_t$ of selecting subspaces. If $\alpha = 0$, according to the importance weight equation,

$$P_{t,\omega_i} = \frac{e^{\alpha \hat{S}_{t-1,\omega_i}}}{\sum_{j=1}^k e^{\alpha \hat{S}_{t-1,\omega_j}}}. \tag{B.1}$$

$P_{t,\omega_i} = e^{\alpha \hat{S}_{t-1,\omega_i}} / \sum_{j=1}^k e^{\alpha \hat{S}_{t-1,\omega_j}} = e^{0 \hat{S}_{t-1,\omega_i}} / \sum_{j=1}^k e^{0 \hat{S}_{t-1,\omega_j}} = 1/k$. If α is large or even close to positive infinity, $P_{t,\omega_i} = e^{\alpha \hat{S}_{t-1,\omega_i}} / \sum_{j=1}^k e^{\alpha \hat{S}_{t-1,\omega_j}} = 1 / \sum_{j=1}^k e^{\alpha(\hat{S}_{t-1,\omega_j} - \hat{S}_{t-1,\omega_i})} \approx 0$ (except for the subspace ω_j with the best reward, $P_{t,\omega_j} \approx 1$). Therefore, if α is too large or small, excluding the subspace with $P_{t,\omega_i} \approx 1$, the probability of selecting each subspace will be the same, and AvgHExp3 loses its meaning. Moreover, GVAE-ABGEP will be close to classical GEPs.

From the perspective of balancing the subspace exploration and the subspace exploitation, the expected regret $R_n(\pi, r_t)$ of AvgHExp3 should be as small as possible. For determining α , we try to use α to limit the upper bound

of $R_n(\pi, r_t)$ [?]. According to Theorem 2, the optimal value of parameter α is $\sqrt{\frac{\log k + n(n-1)}{2nk}}$.

Theorem 2. *For limiting the upper bound of the regret reward $R_n(\pi, r_t)$, the optimal value of α is $\sqrt{\frac{\log k + n(n-1)}{2nk}}$ when $\alpha \leq t$.*

Proof. The following proof is similar to the proof of Theorem 11.1 in [?]. However, the difference between Theorem 11.1 [?] and Theorem 2 is the estimated reward $\hat{R}$. The classical Exp3 [?] uses the total estimated reward while AvgHExp3 uses the average estimated reward.

For any subspace ω_i , the regret after the round n is

$$R_{n,\omega_i} = \sum_{t=\alpha}^{n+\alpha} R_{t,\omega_i} - E \left[\sum_{t=\alpha}^{n+\alpha} R_t \right]. \quad (\text{B.2})$$

According to Equation A.1 and the definition of $\hat{S}'_{t-1,\omega_i}$, i.e., $\hat{S}'_{t-1,\omega_i} = \frac{1}{t-1} \sum_{s=1}^{t-1} \hat{R}_{s,\omega_i}$, $\sum_{t=\alpha}^{n+\alpha} R_{t,\omega_i} = E \left[\sum_{t=\alpha}^{n+\alpha} \hat{R}_{t,\omega_i} \right] = E \left[n \hat{S}'_{n,\omega_i} \right]$. Meanwhile, according to the definition of expectation, $E[R_t]$ is expanded into $E[R_t] = \sum_{i=1}^k P_{t,\omega_i} R_{t,\omega_i} = \sum_{i=1}^k P_{t,\omega_i} E[\hat{R}_{t,\omega_i}]$. So Equation B.2 can be computed as the following equation.

$$\begin{aligned} R_{n,\omega_i} &= \sum_{t=\alpha}^{n+\alpha} R_{t,\omega_i} - E \left[\sum_{t=\alpha}^{n+\alpha} R_t \right] \\ &= n E \left[\hat{S}'_{n,\omega_i} \right] - E \left[\sum_{t=\alpha}^{n+\alpha} \sum_{i=1}^k P_{t,\omega_i} E \left[\hat{R}_{t,\omega_i} \right] \right] \\ &= n E \left[\hat{S}'_{n,\omega_i} - \frac{1}{n} \sum_{t=\alpha}^{n+\alpha} \sum_{i=1}^k P_{t,\omega_i} \hat{R}_{t,\omega_i} \right] \end{aligned} \quad (\text{B.3})$$

For the following inequality

$$e^{\alpha \hat{S}'_{n,\omega_i}} \leq \sum_{j=1}^k e^{\alpha \hat{S}'_{n,\omega_j}}, \quad (\text{B.4})$$

the right side of Equation B.4 is expanded into the form of continuous multiplication in Inequality B.5. Since $\hat{S}'_{0,\omega_j} = 0$ at the initialization of AvgHExp3,

$e^{\alpha \hat{S}'_{0,\omega_j}} = 1$. So, Inequality B.4 is finally transformed into Inequality B.5.

$$\begin{aligned} e^{\alpha \hat{S}'_{n,\omega_i}} &\leq \sum_{j=1}^k e^{\alpha \hat{S}'_{0,\omega_j}} \frac{\sum_{j=1}^k e^{\alpha \hat{S}'_{1,\omega_j}}}{\sum_{j=1}^k e^{\alpha \hat{S}'_{0,\omega_j}}} \cdots \frac{\sum_{j=1}^k e^{\alpha \hat{S}'_{n,\omega_j}}}{\sum_{j=1}^k e^{\alpha \hat{S}'_{n-1,\omega_j}}} \\ &\leq k \prod_{t=1}^n \frac{\sum_{j=1}^k e^{\alpha \hat{S}'_{t,\omega_j}}}{\sum_{j=1}^k e^{\alpha \hat{S}'_{t-1,\omega_j}}} \end{aligned} \quad (\text{B.5})$$

According to the definition of $\hat{S}'_{t,\omega_j}$ and Equation B.1, the continuous multiplication term in Inequation B.5 can be simplified to

$$\begin{aligned} \frac{\sum_{j=1}^k e^{\alpha \hat{S}'_{t,\omega_j}}}{\sum_{j=1}^k e^{\alpha \hat{S}'_{t-1,\omega_j}}} &= \sum_{j=1}^k \frac{e^{\alpha \frac{t-1}{t} \hat{S}'_{t-1,\omega_j}} \cdot e^{\alpha \frac{1}{t} \hat{R}_{t,\omega_j}}}{\sum_{j=1}^k e^{\alpha \hat{S}'_{t-1,\omega_j}}} \\ &\leq \sum_{j=1}^k \frac{e^{\alpha \hat{S}'_{t-1,\omega_j}} \cdot e^{\alpha \frac{1}{t} \hat{R}_{t,\omega_j}}}{\sum_{j=1}^k e^{\alpha \hat{S}'_{t-1,\omega_j}}} \\ &= \sum_{j=1}^k P_{t,\omega_j} e^{\frac{\alpha}{t} \hat{R}_{t,\omega_j}} \end{aligned} \quad (\text{B.6})$$

Since $e^x \leq n + \frac{1}{n}x + 2x^2$ when $x \leq 1$, and according to the following Equation

$$\hat{R}_{t,\omega_j} = 1 - \frac{r_{t,a_t}}{P_{t,\omega_j}} = 1 - \frac{\Pi \{a_t = i\} r_{t,a_t}}{P_{t,\omega_j}} \quad (\text{B.7})$$

where $r_{t,a_t} \geq 0$, so $\frac{\alpha}{t} \hat{R}_{t,\omega_j} \leq 1$, the right side of Equation B.6 can be bounded as

$$\sum_{j=1}^k P_{t,\omega_j} e^{\frac{\alpha}{t} \hat{R}_{t,\omega_j}} \leq n + \frac{1}{n} \sum_{j=1}^k P_{t,\omega_j} \frac{\alpha}{t} \hat{R}_{t,\omega_j} + 2 \sum_{j=1}^k P_{t,\omega_j} \frac{\alpha^2}{t^2} \hat{R}_{t,\omega_j}^2. \quad (\text{B.8})$$

Since $1 + x \leq e^x$ when $x \in R$, Inequality B.8 can be further rewritten as

$$\sum_{j=1}^k P_{t,\omega_j} e^{\frac{\alpha}{t} \hat{R}_{t,\omega_j}} \leq e^{n-1+\frac{1}{n} \sum_{j=1}^k P_{t,\omega_j} \frac{\alpha}{t} \hat{R}_{t,\omega_j} + 2 \sum_{j=1}^k P_{t,\omega_j} \frac{\alpha^2}{t^2} \hat{R}_{t,\omega_j}^2} \quad (\text{B.9})$$

Combining Inequality B.5, Inequality B.6 and Inequality B.9,

$$\begin{aligned} e^{\alpha \hat{S}'_{n,\omega_i}} &\leq k e^{n(n-1) + \frac{1}{n} \alpha \sum_{t=1}^n \sum_{j=1}^k \frac{1}{t} P_{t,\omega_j} \hat{R}_{t,\omega_j} + 2\alpha^2 \sum_{t=1}^n \sum_{j=1}^k \frac{1}{t^2} P_{t,\omega_j} \hat{R}_{t,\omega_j}^2} \\ &\leq k e^{n(n-1) + \frac{1}{n} \alpha \sum_{t=1}^n \sum_{j=1}^k P_{t,\omega_j} \hat{R}_{t,\omega_j} + 2\alpha^2 \sum_{t=1}^n \sum_{j=1}^k P_{t,\omega_j} \hat{R}_{t,\omega_j}^2} \end{aligned} \quad (\text{B.10})$$

Taking logarithm on both sides of Equation B.10 and dividing by α ,

$$\hat{S}'_{n,\omega_i} - \frac{1}{n} \sum_{t=1}^n \sum_{j=1}^k P_{t,\omega_j} \hat{R}_{t,\omega_j} \leq \frac{\log k}{\alpha} + \frac{n(n-1)}{\alpha} + 2\alpha \sum_{t=1}^n \sum_{j=1}^k P_{t,\omega_j} \hat{R}_{t,\omega_j}^2 \quad (\text{B.11})$$

According to $E \left[\sum_{t=1}^n \sum_{j=1}^k P_{t,\omega_j} \hat{R}_{t,\omega_j}^2 \right] \leq nk$ in [?], The expectation of Inequality B.11 can be further modified as

$$E \left[\hat{S}'_{n,\omega_i} - \frac{1}{n} \sum_{t=1}^n \sum_{j=1}^k P_{t,\omega_j} \hat{R}_{t,\omega_j} \right] \leq \frac{\log k}{\alpha} + \frac{n(n-1)}{\alpha} + 2\alpha nk \quad (\text{B.12})$$

Combining Equation B.3 and Inequality B.12,

$$R_{n,\omega_i} \leq \frac{n \log k + n^2(n-1)}{\alpha} + 2\alpha n^2 k. \quad (\text{B.13})$$

According to the inequality $a + b \geq 2\sqrt{ab}$, the right side of Inequality B.13 is

$$\frac{n \log k + n^2(n-1)}{\alpha} + 2\alpha n^2 k \geq 2\sqrt{2n^2 k (n \log k + n^2(n-1))}. \quad (\text{B.14})$$

For limit the right side of Inequality B.13,

$$\frac{n \log k + n^2(n-1)}{\alpha} = 2\alpha n^2 k. \quad (\text{B.15})$$

So the optimal value of α is $\sqrt{\frac{\log k + n(n-1)}{2nk}}$. $\square$